Gauge reader v1 service report: what we built and how it differs from v0

Technical edition · Oct 13, 2026 · engine 1.0.0, commit `d510b11` · report schema 1.0

v1 reads the water level of a paddy-field gauge from a drone photo. It runs as a Docker container: a zip of photos goes into one folder and a JSON report comes out of another. On the 127-photo pilot batch, v1 gives an accepted, exactly correct stripe count on **97 photos (76.4%)** against **19 (15.0%)** for the existing reader, v0. The target is 70–80%. This figure comes from the same photos v1 was developed on; a fresh, unseen batch is needed before it can be called validated accuracy.

15.0%	**76.4%**	**28 → 16**	**127 / 127**
v0: accepted and exact (19 of 127)	v1: accepted and exact (97 of 127)	accepted but wrong, v0 → v1	container = research code, same reading and decision

System specification

Input	A zip of drone photos (jpg, jpeg, png), one folder per visit named `<checkpoint> - <ddmmyyHHMMSS>`. Loose photos are accepted and read on their own.
Output	`out/<zip>.report.json`: summary, one entry per visit, one entry per photo (JSON Schema draft 2020-12, `schema_version` 1.0). A zip that cannot be read gives `out/<zip>.error.json`.
Runtime	Docker (Linux, amd64). Inside the image: Python 3.11, NumPy 1.26.4, SciPy 1.13.1, Pillow 12.1.1, OpenCV (headless) 4.10.0.84. No GPU, no network access, no paid service.
Architecture	One container: a folder watcher, a batch reader that opens the zip in place, and the reading engine (rod finder, stripe-rhythm reader, confidence model, client rules).
Resources	2 GB memory limit; peak 518–528 MB on the pilot zip; one CPU core per worker; image 709 MB on disk.
Validation scope	127 photos processed; all 127 in the accuracy comparison (exact count). The client sent 128; one is missing from our copy. Development dataset only.
Status	Built, unit-tested and run end to end on the pilot zip. Not yet run on photos it has never seen. Centimetres per stripe not yet confirmed.

1. What we did

The client flies a drone over each paddy field every day and photographs a float gauge in the water. Red stripes come up out of a black sleeve as the water rises, about one stripe per centimetre; no stripes means the field is dry. Crop often grows over the gauge, and the client's existing reader (v0) gets only about half of the photos right. We were asked to reach 70–80% without changing the gauge.

1. **Answer key.** Every pilot photo has a hand count of its stripes (section 6 describes how it was made and checked).
2. **v0 rebuilt and measured.** v0's code was not available, so it was rebuilt rule by rule from its written description and run on the same photos.
3. **v1 designed.** v1 measures the stripe rhythm instead of counting bars, then a small trained model estimates how likely the count is to be exactly right, and the client's acceptance rule decides. Each idea was tried separately and kept only if it helped on fields the model had not seen.
4. **Weak spots checked.** Enlarging far gauges, adding a stripe at the sleeve and other ideas were tried and dropped because they did not help (section 7).
5. **Packaged as a service** and **tested with the real zip** (127 photos, 713 MB): the container gives the research code's answer on every photo.

2. Architecture and deployment

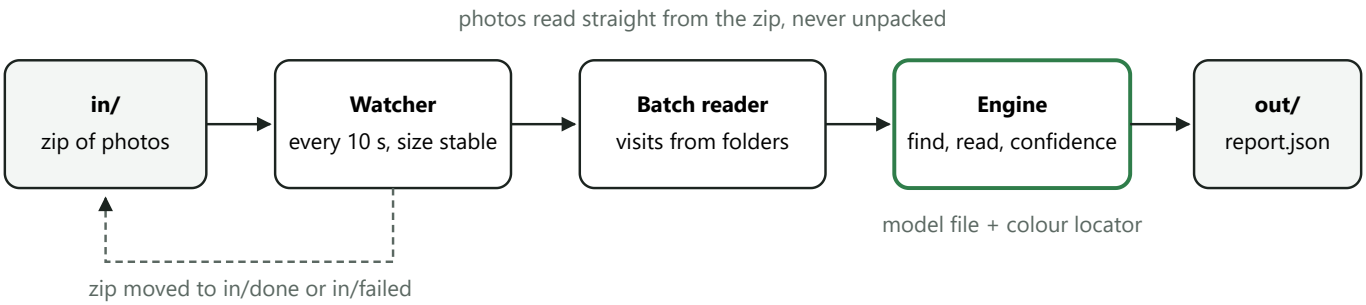


Figure 1. One container. The engine is the same code the research results come from.

Directory layout and naming

```
service/                          docker compose up -d --build   (run here)
  in/                             drop zips here (created on first start)
    done/                         zips that were read, or were already read
    failed/                       zips that could not be read
  out/
    <zip>.report.json             one report per zip
    <zip>.error.json              why a zip failed
    .processed.json               fingerprints of zips already read

expected zip:
  way points/                                      (any top folder, or none)
    AIT-CP-5286-2931-CKP K26 NG - 011026101600/    <checkpoint> - <ddmmyyHHMMSS>
      dji_fly_..._photo.jpg                        2 shots per visit
      dji_fly_..._photo.jpg
```

Behaviour	Exact rule
When a zip is read	Every `POLL_SECONDS` the watcher lists `in/*.zip`. A zip is read only when its size is non-zero and the same as on the previous look, so a copy in progress is never read.
Fingerprint and duplicates	SHA-256 of the zip. Fingerprints of read zips are kept in `out/.processed.json` and survive restarts. A zip with a known fingerprint, under any name, is moved to `in/done` without a new report.
Visits	Photos are grouped by their folder. A folder whose name matches `<checkpoint> - <12 digits>` and holds 1–4 photos is one visit. Loose photos, and folders with more than 4 photos, are read one photo per visit (flag `no_twin_shot`).
Files skipped	Names with an absolute path, a drive letter or `..` (reason `unsafe path`); anything not jpg/jpeg/png (`not a jpg or png`); a photo larger than `MAX_PHOTO_MB` uncompressed (`photo too large`). All are listed under `skipped`.
Zip-level failure	Not a readable zip, no jpg/png inside, or more than `MAX_IMAGES` photos: `out/<zip>.error.json` is written and the zip goes to `in/failed`.
Photo-level failure	A photo that cannot be decoded or read from the zip gets `status: error`; the rest of the batch carries on.
Writing	Reports and error files are written to a `.part` file and renamed, so a half-written report is never visible.
Health check	The watcher writes a heartbeat at every poll and after every visit. `python -m gaugesvc health` passes while the heartbeat is younger than 3 × `POLL_SECONDS` + 120 s (150 s by default). Docker runs it every 30 s (timeout 10 s, start period 30 s, 3 retries).

Configuration (verified against the Dockerfile, the Compose file and the code)

Setting	Default	Effect
`POLL_SECONDS`	10	How often `in/` is checked
`CM_PER_STRIPE`	1.0	Centimetres per stripe for `level_cm` and `soil_state` (an assumption until measured)
`WORKERS`	1	Visits read in parallel. Only 1 worker has been measured; more workers need more memory
`MAX_IMAGES`	5000	Largest zip accepted, in photos
`MAX_PHOTO_MB`	60	Largest single photo accepted (uncompressed size in the zip)
`IN_DIR`, `OUT_DIR`	`/data/in`, `/data/out`	Mounted from `./in` and `./out`
`GIT_COMMIT` (build argument)	`unknown`	Written into every report as `engine.commit`

Setting	Default	Effect
Compose limits	`mem_limit: 2g`, `restart: unless-stopped`, runs as user `gauge` (uid 10001), no ports published	

```
cd service
GIT_COMMIT=$(git rev-parse --short HEAD) docker compose up -d --build    # start (builds the image)
docker compose logs -f                                                    # follow the log
docker compose down                                                       # stop
python -m gaugesvc run some.zip report.json                               # one zip, without Docker
```

3. Interface contract

Report fields

Field	Meaning
`schema_version`, `engine`	`"1.0"`; engine name, version (`1.0.0`) and the commit the image was built from
`zip`, `sha256`, `processed_at`, `seconds`	Input name and fingerprint, finish time, reading time
`settings`	`cm_per_stripe`, `accept_at` (0.7), `guide_zero_cm` (10)
`summary`	Counts: photos, accepted, review, no_gauge, error, visits, skipped
`visits[]`	`folder`, `checkpoint`, `taken` (from the folder name), `photos`, `reading` (mean of the accepted shots, with level and soil state, or `status: review`), `consistent` (accepted shots within 1 stripe)
`photos[]`	`file`, `visit`, `status`, `stripes`, `candidate_stripes`, `dry`, `level_cm`, `level_vs_guide_cm`, `soil_state`, `confidence`, `gauge_box`, `note`, `error`, `flags`
`skipped[]`	`name` and `reason` of every file not read

Status	Meaning	Fields present
`accepted`	Confidence ≥ 0.7 and no bad-gauge flag	stripes, dry, level_cm, level_vs_guide_cm, soil_state, confidence
`review`	A reading exists but confidence < 0.7, or the bad-gauge check fired. No number is reported.	candidate_stripes (for the reviewer), confidence, note
`no_gauge`	Neither the rod finder nor the colour locator found a gauge	stripes: null
`error`	The photo could not be decoded or read from the zip	error

Flags: `no_twin_shot` (read alone), `twin_unreadable` (the other shot gave no reading), `colour_locator_used`, `bad_gauge`. Soil state: level against the guide = `level_cm` − 10; above +2 Flooded, −2 to +2 Saturated, −12 to below

−2 Drying, below −12 Irrigate now.


```
{ "schema_version": "1.0",
  "engine": {"name": "CarbonMint gauge reader", "version": "1.0.0", "commit": "d510b11"},
  "zip": "waypoints.zip", "sha256": "…", "seconds": 236.1,
  "summary": {"photos": 127, "accepted": 111, "review": 16, "no_gauge": 0, "error": 0, "visits": 66,
"skipped": 0},
  "visits": [ { "folder": "…/AIT-CP-2050-9573-CKP K26 VP - 051026113308", "checkpoint": "AIT-CP-2050-9573-
CKP K26 VP",
                "taken": "2026-10-05T11:33:08", "photos": ["…", "…"], "consistent": true,
                "reading": {"status": "accepted", "stripes": 20.0, "level_cm": 20.0, "level_vs_guide_cm":
10.0, "soil_state": "Flooded"} } ],
  "photos": [
    { "file": "…/dji_fly_20261001_101600_0358_…_photo.jpg", "status": "accepted", "stripes": 4, "dry":
false,
      "level_cm": 4.0, "level_vs_guide_cm": -6.0, "soil_state": "Drying", "confidence": 0.934,
      "gauge_box": [2095, 969, 2236, 1071], "flags": [] },
    { "file": "…/dji_fly_20261001_100440_0349_…_photo.jpg", "status": "review", "stripes": null,
      "candidate_stripes": 19, "confidence": 0.456, "note": "review: confidence below 0.7",
      "gauge_box": [1977, 858, 2217, 1559], "flags": [] } ],
  "skipped": [] }

out/broken.error.json:   { "zip": "broken.zip", "sha256": "…", "error": "not a readable zip: BadZipFile" }
```


Real entries from the pilot run, trimmed. The full schema is `service/report.schema.json`; a four-visit real report is `service/examples/sample_report.json`.

4. How one photo is read

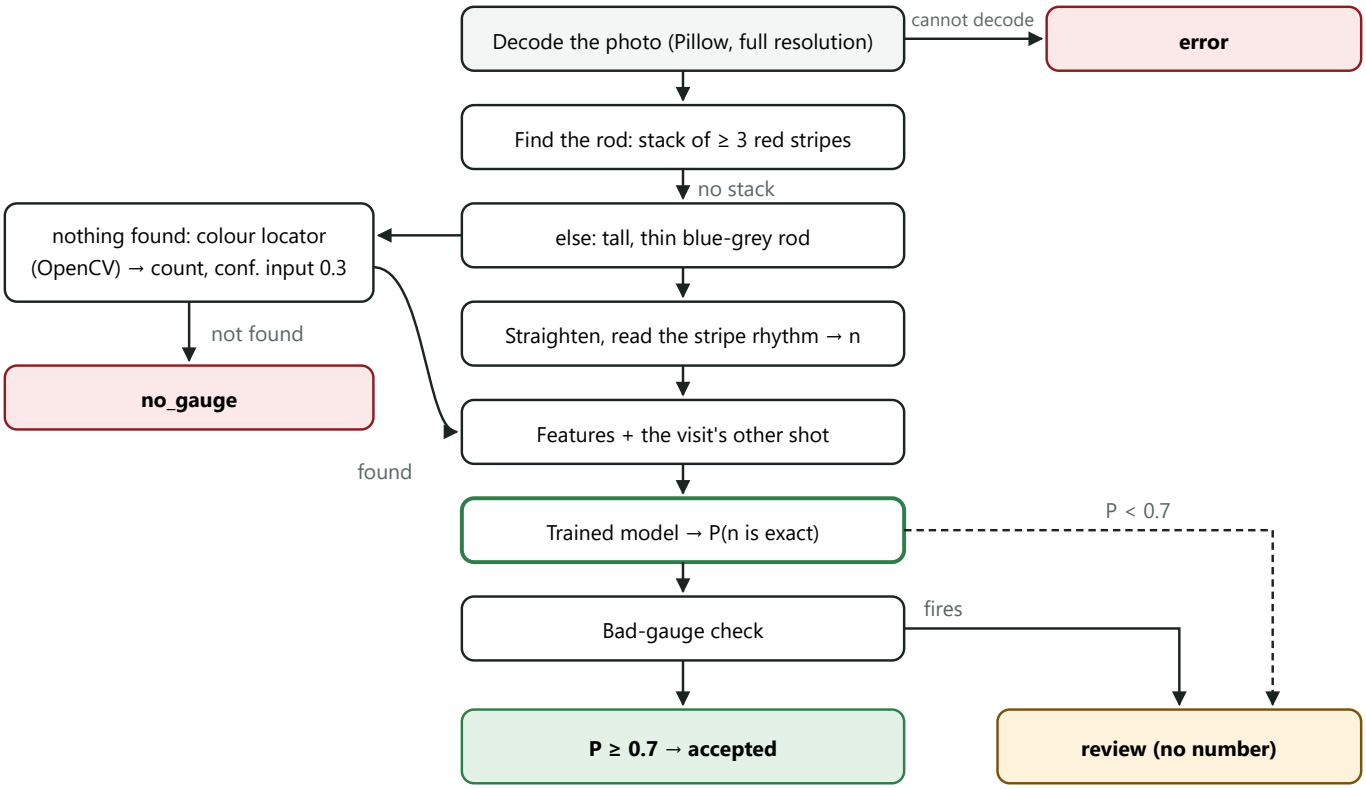


Figure 2. Decision flow, as implemented in `gauge/` and `service/gaugesvc/engine.py`. v1 has no second-look pass: a photo that is not confident enough goes to review.

Step	Exact rule
Stripe pixels	R − max(G, B) > 25, R > 1.25 × max(G, B), R > 40 and B ≥ G − 12 (straw and brown soil, where G is well above B, are excluded). Red is judged against green and blue, not by brightness, so shaded stripes count.
Find the rod	Stripe blobs of ≥ 8 px are grouped vertically; a group needs ≥ 3 separate stripes on a near-vertical axis ($\lvert direction_y \rvert \geq 0.85$, elongated). Pieces split by a leaf are joined when they lie within 2.5 half-widths of the same axis. Largest score wins. With no stripe stack, a tall, thin, near-vertical blue-grey tube (fill ≥ 0.3, length ≥ 3 widths) is taken instead.
Straighten	The ring centre is found row by row in a strip 5 half-widths wide; a quadratic (straight line for short rods) through those centres follows lean and bend, and the rod's core is resampled along it.
No paint showing	Fewer than 4 rows with stripe paint: the reading is 0 stripes (dry), a valid reading.
Stripe gap (pitch)	Autocorrelation of the redness down the rod. The pitch is the **shortest** lag whose peak is at least 0.75 of the strongest, so a stretch of 2–4 stripes is not taken as one gap. With 2–6 clear stripes the median spacing is used instead.
End stripes	Peaks at either end weaker than 0.3 × the median stripe are dropped (sleeve rims, stains).

Step	Exact rule
Count	Gap by gap from the first stripe to the last: each gap counts round(gap ÷ local pitch) stripes, at least 1, where the local pitch is the median of the neighbouring gaps that lie between 0.6 and 1.5 × the pitch. Hidden stripes inside the run are counted; perspective is followed.
Colour locator	Only when nothing above finds a rod: our colour method's locator (brightness evened out first, then plain) finds the gauge by red stripes, blue-grey pipe and black sleeve on a 1000 px copy, and counts bands in its box (no bands = dry). Its reading enters the model with a fixed confidence input of 0.3.
Confidence	Logistic regression (L2 = 1, fitted by Newton steps) on 10 standardised inputs: the reader's own score, pattern strength, share of stripes actually seen, log sharpness, log rod length, red coverage, the other shot reads the same count, the other shot within 1, no other shot, reading is 0. Output: P(count is exactly right).
The other shot	The two shots of a visit are read first; each shot's features include the other's count. A missing or unreadable twin is an input (“no other shot”) and a flag, not a failure.
Bad gauge	More than 25 stripes, or a stripe gap outside 0.45–1.7 × the rod's half-width (with ≥ 3 stripes): never accepted, flag `bad_gauge`, confidence capped below 0.7.
Accept	P ≥ 0.7 (the client's bar). Visit reading = mean of its accepted shots (the client's folder rule); none accepted → the visit stays in review.

5. What the service is built from

Part	Version	Use
Base image	`python:3.11-slim` (3.11.17 at build)	Linux amd64 runtime
NumPy / SciPy	1.26.4 / 1.13.1	Rod finding, straightening, rhythm, confidence model
Pillow	12.1.1	Decoding photos straight from the zip
OpenCV (headless)	4.10.0.84	Colour locator only
Confidence model	`models/v2_confidence.json`, 791 bytes	Weights, means and scales of the logistic model
Folder watcher, batch reader, report	`service/gaugesvc`, engine 1.0.0	Our own code, standard library only
Test-only	jsonschema 4.23.0	Checks reports against the schema

6. Evaluation method

Dataset

127 drone photos (4096 × 3072 JPEG, about 5.6 MB each) from 7 fields, 66 visits, 23 Sep – 6 Oct 2026; the client's pilot batch minus one photo missing from our copy. This is the development dataset: the reading rules were designed while looking at these photos.

Ground truth (`data/labels.csv`)

1. For every photo the gauge was straightened and enlarged (`scripts/label_strips.py`) and the first and last visible stripe marked. The stripes were counted as whole stripes; a stripe partly hidden by the cap or sleeve counts when it is mostly visible. The first read was made by an AI assistant from these enlarged strips. Each label carries a confidence (high 103, low 24) and an occlusion flag (46 photos).
2. A person checked random samples against the photos (`gauge/spotcheck.py`, fixed seed). Round 1, 15 photos: 10 exact, mostly a half-stripe convention; the convention was fixed to whole stripes and 3 labels corrected. Round 2, 15 new photos: 15 of 15 exact.

Photo sets

Each photo was sorted by eye into one set: **Clear** (59), the gauge and its stripes plainly visible; **Partly hidden** (43), leaves or grass across the stripes; **Hard** (25), the gauge barely visible even to the eye.

Metrics

Accepted and exact (primary)	Photos with status accepted whose stripe count equals the label, divided by all 127 photos
Accepted but wrong	Accepted photos whose count differs from the label by any amount
Right when accepted	Accepted and exact ÷ accepted
Review, no gauge, error	Shares of photos with those statuses
Visits	A visit counts when the mean of its accepted shots is within 0.5 of the visit's median label

Leave one field out. The confidence model is scored field by field: for each of the 7 fields it is trained on the other 6 and applied to that field, so no photo is scored by a model that saw its field. The figures below use this. The deployed service uses one model trained on all 7 fields, so on these same photos it looks slightly better (111 accepted, 98 exact); that in-sample figure is not quoted as accuracy. Reproduce: `python -m gauge.run`, then `python scripts/compare.py` (writes `docs/comparison.md`).

> The 76.4% is measured on the development dataset. It shows the method works on these fields; it is not independent validation. A fresh labelled batch from fields and days v1 has never seen is needed to estimate real-world accuracy.

7. Results: how v1 differs from v0

	v0	v1
Approach	Pinkest column, counts the bars in it	Stripe rhythm from the first stripe to the last
How sure it is	How regular the stripes look	Trained chance that the count is exactly right
Shade	Needs bright pink (value ≥ 50%); shaded stripes missed	Red against green and blue; dark stripes count
A leaf across the gauge	Keeps the longest unbroken run; the rest is dropped	Hidden stripes inside the run are counted
A dry gauge	No stripes, no reading	0 stripes is a reading (28 of 29 dry photos right)
Second shot, broken gauge, unsure photo	Not used / not checked / left out	Confidence input / flagged / sent to review with the count to check

Photos	v0 accepted & exact	v1 accepted & exact	v1 accepted, wrong	v1 review (right count / wrong)	v1 no gauge
Clear (59)	16 = 27.1%	45 = 76.3%	5	9 (5 / 4)	0
Partly hidden (43)	3 = 7.0%	32 = 74.4%	9	2 (2 / 0)	0
Hard (25)	0 = 0%	20 = 80.0%	2	3 (1 / 2)	0
All (127)	**19 = 15.0%**	**97 = 76.4%**	**16**	**14 (8 / 6)**	**0**

v0: 47 accepted, 28 of them wrong; 66 photos with no reading. v1: 113 accepted (89.0%), right when accepted 85.8%; 0 photos with errors. Visits: v0 9 of 66, v1 50 of 66. By field (leave one out), v1 ranges from 47.4% to 100% accepted and exact; the weakest are AIT-CP-2050-9573 (9 of 19) and GAN-CP-3116-8972 (10 of 17).

Why v0 misses 108 photos. No gauge column or stripe stack found, mostly shaded stripes below its brightness bar: 37. Dry gauge, no reading possible: 29. Accepted with a wrong count, mostly leaves breaking its run: 28. Wrong count, not accepted: 13. Right count under its confidence bar: 1.

Tried and dropped, each scored the same way: Lanczos 2–3× enlarging of small rods (no change on 13 rods), counting an extra stripe at the sleeve (+3, −4), following a faint rhythm past the last stripe (+7, −6), extra confidence inputs (lower), and using the colour method's count or score as a vote (lower). Under the looser rule used in some pilot reports (±1, or ±3 on estimated tall stems; 9 uncountable photos left out), v1 scores 94 of 118 (79.7%); see the companion note.

8. Failure analysis

All 16 accepted-but-wrong photos, grouped. “Confirmed” means the cause was seen on the straightened strip of the photo; “suspected” means it fits the evidence but was not traced step by step. 7 of the 16 come from one field, GAN-CP-3116-8972.

Observed issue	Photos	Likely cause	Status	Impact	Proposed investigation
One stripe short at the sleeve (16→15, 15→14)	4	A thin last stripe at the sleeve is weaker than 0.3 × the median and is dropped as a sleeve rim	confirmed	1 cm low	Measure band width as well as redness at the ends
One stripe extra (24→25)	2	A leaf splits one stripe into two peaks	suspected	1 cm high	Reject gaps under 0.6 × local pitch
Two extra on a leaf-crossed stem (23→25)	1	Same as above, twice	suspected	2 cm high	As above
Leaves hide the top of the stem (22→19), both shots	2	Stripes under leaves above the first visible stripe cannot be counted; label marked low confidence	confirmed	3 cm low	Penalise a long grey cap above the first stripe
Blurred short stems (4→0/1, 6→1/3) accepted at 0.99	4	Blur merges stripes; the model trusts a small rod where every visible stripe was “seen”	suspected	3–5 cm low; a wet field can read dry	Add sharpness relative to rod size to the model; re-score leave-one-field-out
Far shot, washed-out pink (24→13), both shots	2	Distant pale stripes fail the stripe-pixel test; the rhythm continues past the detected end	confirmed	11 cm low	Relative threshold along the rod; capture guide (fly closer)
Dry gauge read as 2	1	A tiny red patch (rod 58 px) taken as a two-stripe stack	suspected	2 cm high	Minimum rod size for a stripe stack

Operational case	What happens today	Status
Faded stripes	Pale but red stripes count; very washed-out pink on far shots is missed (above)	partly handled
Short stems	Read directly; blur is the risk (above)	partly handled
Crop over the gauge	Stripes hidden inside the run are counted; stripes hidden at the top are not	partly handled
Missing twin shot	Read with one input fewer, flagged; 6 photos in the pilot	handled
Faulty gauge	Bad-gauge rule exists; 0 false alarms on the 127 good gauges; never seen a real faulty gauge	untested

Operational case	What happens today	Status
Invalid zips	Broken, empty, photo-less and oversized zips go to `failed` with an error file (tested)	handled
Soil state for a dry gauge	0 stripes → −10 cm → Drying, never Irrigate now, by v0's own rule	needs client decision

9. Performance and resource profile

Hardware	Laptop: Intel Core i5-1155G7 (4 cores, 8 threads), 15.8 GB RAM, Windows 11 Home
Container platform	Docker 29.6.2 on Docker Desktop (WSL2, Linux kernel 6.18); 8 CPUs and 7.65 GiB visible to Docker; container limited to 2 GB; `WORKERS=1`
Input	Pilot zip: 127 photos, 4096 × 3072 JPEG, 713 MB, 66 visit folders
Measurement	**Reading time**: the report's `seconds` field, from opening the zip to the finished report in memory (excludes the copy, the wait for a stable size and writing the file). **Wall time**: from starting the copy into `in/` to the report file appearing, checked every 5 s (includes copying 713 MB and up to two polls). **Memory**: the container's `memory.peak` since start, including idle.

Run	Reading time	Wall time	Per photo	Peak memory	Result
Oct 12, run 1	267.0 s	292 s	2.1 s	518 MB	111 accepted, 16 review, 0 no gauge, 0 error
Oct 12, run 2 (commit `d510b11`)	236.1 s	263 s	1.9 s	528 MB	same, 127/127 parity, schema valid

Start-up: the container reported healthy 12–15 s after start. Storage: a report is about 94 KB for 127 photos; the zip itself is kept in `in/done`, so allow at least the zip's size again. Two runs on one machine are an indication, not a guarantee; other hardware, larger photos or more workers will change the figures.

10. Testing and reproducibility

- **Parity with the research code:** `python service/tests/parity.py service/out/waypoints.report.json` → same reading 127/127, same accept decision 127/127.
- **Service tests** (`python -m unittest discover -s service/tests`, 8 tests): visit grouping, damaged photo, hostile names (`../`, drive letters), non-photo files, loose photos, folders of more than 4, empty / photo-less / broken / oversized zips, soil rules, visit rule, report validated against the schema, partial copy, duplicate under another name, broken zip to `failed` with its fingerprint, fingerprints kept across restarts.
- **Reader tests** (`python -m unittest tests/test_gauge.py tests/test_app.py`, 31 tests): synthetic gauges with known answers, hidden stripes, a pitch that the old rule took at 4× the true gap, dry gauges, the confidence model, the bad-gauge rule, the colour-locator fallback.

- **Live checks:** real zip end to end twice; the same zip dropped again under another name was recognised and skipped; container healthy, non-root, restart policy set.
- **Versions:** commit `d510b11`; Python 3.11.17 in the image; packages pinned in `service/requirements.txt`; ground truth `data/labels.csv`; leave-one-field-out results `data/v2_reads.csv`.

11. Limits and engineering backlog

Priority	Item	Acceptance criterion
1	Run v1 on a fresh labelled batch	≥ 100 new photos from fields or days not in the pilot, labelled the same way; accepted-and-exact and accepted-but-wrong reported with a confidence interval
1	Confirm centimetres per stripe	Three gauges measured in the field; `CM_PER_STRIPE` set; level within ±1 cm on those gauges
1	Agree the soil state for a dry gauge	Client rule written down; a test fixes the output for 0 stripes
2	Stop confident errors on blurred short stems	The 4 known photos fall below 0.7 with no more than 2 correct accepts lost, leave-one-field-out
2	Read washed-out far shots	The GAN-CP-3116-8972 27 Sep pair reads 24 or goes to review; no new accepted-but-wrong
2	Person reviews all 16 accepted-but-wrong photos	Each marked reader error or label error; key and figures updated
3	Heartbeat with `WORKERS` > 1	Health stays “healthy” through a 1,000-photo zip with `WORKERS=2` (today the heartbeat is written per visit only with one worker)
3	Bad-gauge check on a real faulty gauge	A photo of a deliberately faulty gauge is flagged and not accepted

12. Handover checklist

8 of 12 verified. Open items are in the backlog above.

Area	Item	Status	Evidence
Reproducibility	Source commit or release recorded	verified	Commit `d510b11` in every report
	Python, Docker and dependency versions recorded	verified	Sections 5 and 9
	Benchmark command and configuration documented	verified	Sections 6, 9 and 10
	Ground truth and metric definitions provided	verified	Section 6, `data/labels.csv`

Area	Item	Status	Evidence
Validation	Fresh dataset not used in development evaluated	open	No new photos yet
	Stripe-to-centimetre calibration verified	open	Needs a field measurement
	Dry-gauge soil-state rule confirmed	open	Needs the client
	All accepted-but-incorrect cases reviewed	open	Classified in section 8; not yet reviewed by a person
Deployment	Start-up, configuration and shutdown documented	verified	Section 2
	Health checks, logs and failed-zip handling verified	verified	Live run and tests
	Duplicate uploads and interrupted copies tested	verified	Tests and live duplicate drop
	Storage, memory and processing requirements documented	verified	Section 9